NoGuessMethod — Project Documentation

What This Project Is

NoGuessMethod is a structured training web app for intermediate lifters. It delivers a daily rotating Push/Pull/Legs/Core/Recovery program with premium unlocks (form cues, progression rules, nutrition briefs). Members sign up for free; premium is $19.99/mo via Stripe.

Tech Stack

Layer	Tool
Frontend	React 18 + Vite 7
Routing	React Router v6
Animations	Framer Motion v12
Auth + Database	Supabase (JS v2)
Backend Functions	Netlify Functions (Node.js)
Payments	Stripe (Checkout + Billing Portal)
SMS Reminders	Twilio (optional)
Hosting	Netlify

Project Structure

```
noguessmethod_login_logout_nav_fixed/
├── index.html                  # Vite entry point
├── vite.config.js              # Vite + React plugin config
├── package.json                # Dependencies
├── netlify.toml                # Netlify build + redirect rules
├── .env                        # Environment variables (never commit)
├── .gitignore
│
├── assets/
│   └── style.css               # Global stylesheet (all styles live here)
│
├── public/
│   └── assets/
│       ├── ngm-logo-square.jpeg
│       ├── ngm-logo-banner.jpeg
│       └── favicon.png
│
├── src/
│   ├── main.jsx                # App entry — mounts React, imports CSS
│   ├── App.jsx                 # Router with AnimatePresence for transitions
```

```
│   │
│   ├── context/
│   │   └── AuthContext.jsx     # Global auth state (session, signOut)
│   │
│   ├── lib/
│   │   └── supabase.js         # Supabase client instance
│   │
│   ├── data/
│   │   └── workouts.js         # 8 workout types, 30-day schedule, helpers
│   │
│   ├── components/
│   │   ├── Header.jsx          # Nav with hamburger for mobile
│   │   ├── Footer.jsx          # Site footer
│   │   ├── PageTransition.jsx  # Framer Motion wrapper for route transitions
│   │   └── ProtectedRoute.jsx  # Redirects to /login if no session
│   │
│   └── pages/
│       ├── Home.jsx            # Landing page + waitlist form
│       ├── Login.jsx           # Email/password sign in
│       ├── Signup.jsx          # Account creation + profile insert
│       ├── Account.jsx         # Member hub, subscription status, portal link
│       ├── Workout.jsx         # Daily workout with premium gating
│       ├── Settings.jsx        # Profile, training, reminders, password
│       ├── Admin.jsx           # Admin dashboard (members, posts, waitlist)
│       ├── Upgrade.jsx         # Pricing + Stripe checkout
│       ├── Success.jsx         # Post-payment confirmation
│       ├── Free.jsx            # Free board submission form
│       ├── Investors.jsx       # Investor contact form (Formspree)
│       └── NotFound.jsx        # 404 page
│
└── netlify/
    └── functions/
        ├── create-checkout.js        # POST → Stripe checkout session
        ├── create-portal.js          # POST → Stripe billing portal
        ├── stripe-webhook.js         # Stripe webhook → update profiles table
        ├── admin-data.js             # GET → all members, posts, waitlist
        ├── admin-set-subscription.js # POST → manually set member tier
        ├── admin-update-post.js      # POST → approve/resolve free board post
        └── send-reminders.js         # Scheduled → Twilio SMS reminders
```

What Was Changed and How

1. Converted from Vanilla HTML to React (Vite)

The original project was a collection of plain `.html` files with inline scripts. Every page was rewritten as a React component maintaining the same CSS class names and visual structure.

How: Created `vite.config.js` with `@vitejs/plugin-react`, updated `index.html` to use `<div id="root">` + Vite module script, created `src/main.jsx` as the entry point. All 12 HTML pages became `.jsx` components under `src/pages/`.

Why Vite 7 not 8: `@vitejs/plugin-react@4.x` only supports Vite `^4–7`. Vite 8 was pinned in the original package.json and caused an install conflict — downgraded to `^7.0.0`.

---

2. React Router v6 SPA Routing

All `<a href>` navigation replaced with React Router `<Link>` and `useNavigate`. A single `BrowserRouter` wraps everything in `App.jsx`.

Netlify SPA catch-all in `netlify.toml`:

```
[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

This ensures direct URL visits (e.g. `/account`) don't 404 on Netlify.

---

3. Supabase Auth + Database

Client (`src/lib/supabase.js`):

```
import { createClient } from '@supabase/supabase-js'
export const supabase = createClient(SUPABASE_URL, SUPABASE_ANON_KEY)
```

AuthContext (`src/context/AuthContext.jsx`) wraps the whole app and exposes `session` + `signOut` via `useAuth()`. It subscribes to `onAuthStateChange` so every component reacts to login/logout automatically.

ProtectedRoute (`src/components/ProtectedRoute.jsx`) — renders `null` while auth loads, redirects to `/login` if no session, otherwise renders children.

Signup fix: `emailRedirectTo` was missing — without it, confirmation emails link back to Supabase's domain instead of the app. Fixed to `${window.location.origin}/account`.

---

4. Netlify Functions (Backend)

All backend logic lives in `netlify/functions/`. They're called at `/api/*` which Netlify redirects to `/.netlify/functions/*`.

Each function receives an `Authorization: Bearer <supabase_access_token>` header, verifies the user with the Supabase service role key, and performs the requested operation.

Function	What It Does
`create-checkout`	Creates a Stripe Checkout session for $19.99/mo
`create-portal`	Opens the Stripe Billing Portal for an existing subscriber
`stripe-webhook`	Handles `checkout.session.completed` and `customer.subscription.deleted` — updates `profiles.subscription` in Supabase
`admin-data`	Returns all members, free board posts, and waitlist signups (admin only)

`admin-set-subscription`	Manually grants or revokes Premium for a member (admin only)
`admin-update-post`	Sets a free board post's status to `approved` or `resolved` (admin only)
`send-reminders`	Runs on a schedule — queries members with a phone number + preferred time and sends an SMS via Twilio

---

5. Page Transitions (Framer Motion)

`PageTransition.jsx` is a thin wrapper using Framer Motion's `motion.div`:

- Enters: fade in + slide up 18px → 0
- Exits: fade out + slide up 12px
- Easing: `[0.16, 1, 0.3, 1]` (expo out) matching the CSS `--ease-out-expo` variable

`App.jsx` uses `AnimatePresence mode="wait"` with `useLocation` to key each route, so the exit animation completes before the next page enters.

Every page's root element was changed from `<>` to `<PageTransition>`.

---

6. Mobile Hamburger Navigation

`Header.jsx` now manages an `open` state. On screens ≤860px the desktop `<nav>` hides and a `.hamburger` button appears.

- Clicking toggles the `.mobile-nav` full-screen overlay
- Body scroll locks while the menu is open (`document.body.style.overflow = 'hidden'`)
- Menu closes automatically on any link click or route change (`useEffect` on `location.pathname`)
- Three `<span>` bars animate to an X via CSS when `.open` is applied

---

7. Styling Improvements (`assets/style.css`)

- CSS variable `--ease-out-expo: cubic-bezier(0.16, 1, 0.3, 1)` for consistent easing
- All buttons, nav links, and cards have `transition` on transform/background/box-shadow
- Hover: `translateY(-1px)` lift; active: `scale(0.97)` press
- Primary button hover glow: `box-shadow: 0 6px 28px rgba(255,255,255,.22)`
- Input focus ring: `box-shadow: 0 0 0 3px rgba(255,255,255,.07)`
- `.fade-in` / `.fade-in.visible` utility for scroll-reveal (add via IntersectionObserver if needed)
- `@keyframes pulse` loading state
- Responsive breakpoints at 860px (hamburger appears) and 520px (layout adjustments)

Supabase Database Schema (Must Run)

Before the app works, run this SQL in your Supabase project under **SQL Editor**:

```
-- Profiles table (linked to Supabase Auth users)
create table if not exists profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  username text,
  role text default 'member',
```

```
  subscription text default 'free',
  stripe_customer_id text,
  training_level text,
  goal text,
  phone_number text,
  preferred_time text,
  created_at timestamptz default now()
);

-- Free board submissions
create table if not exists free_posts (
  id uuid primary key default gen_random_uuid(),
  name text,
  email text,
  board text,
  message text,
  status text default 'pending',
  created_at timestamptz default now()
);

-- Beta / waitlist signups
create table if not exists beta_users (
  id uuid primary key default gen_random_uuid(),
  name text,
  email text,
  training_level text,
  goal text,
  created_at timestamptz default now()
);

-- Row Level Security
alter table profiles enable row level security;
create policy "Users can read own profile" on profiles for select using (auth.uid() = id);
create policy "Users can update own profile" on profiles for update using (auth.uid() = id);

alter table free_posts enable row level security;
create policy "Anyone can insert posts" on free_posts for insert with check (true);

alter table beta_users enable row level security;
create policy "Anyone can insert to waitlist" on beta_users for insert with check (true);
```

Environment Variables

Create a `.env` file in the project root (already in `.gitignore` — never commit this):

```
# Supabase
SUPABASE_URL=https://ifimawfcqooucjktmcop.supabase.co
SUPABASE_SERVICE_ROLE_KEY=<your service role key from Supabase dashboard>

# Stripe (needed for payments)
```

```
STRIPE_SECRET_KEY=sk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...
STRIPE_PRICE_ID=price_...

# Twilio (optional — for SMS reminders)
TWILIO_ACCOUNT_SID=AC...
TWILIO_AUTH_TOKEN=...
TWILIO_PHONE_NUMBER=+1...
```

The frontend Supabase anon key is hardcoded in `src/lib/supabase.js` (safe — it's a public key with RLS enforcing access). The service role key is only used server-side inside Netlify Functions.

Running Locally

Prerequisites

- Node.js 18+
- npm
- Netlify CLI (`npm install -g netlify-cli`)

Steps

```
# 1. Install dependencies
npm install

# 2. Start the dev server (frontend only — no backend functions)
npm run dev

# 3. OR start with Netlify (includes /api/* functions)
netlify dev
```

- `npm run dev` → `http://localhost:5173` (or next available port)
- `netlify dev` → `http://localhost:8888` (recommended — runs functions too)

If you just need to browse the UI without Stripe/Twilio, `npm run dev` is fine.

Deploying to Netlify

Option A — Netlify CLI

```
# Build and deploy
netlify build
netlify deploy --prod
```

Option B — GitHub + Netlify Dashboard

1. Push the project to a GitHub repository
2. Go to netlify.com → **Add new site** → **Import from Git**
3. Select your repo
4. Build settings are auto-detected from `netlify.toml` :

- Build command: `vite build`
- Publish directory: `dist`

5. Add all environment variables under **Site settings → Environment variables**
6. Deploy

Stripe Webhook (Required for payments to activate Premium)

After deploying, go to Stripe Dashboard → Webhooks:

1. Add endpoint: `https://your-site.netlify.app/api/stripe-webhook`
2. Select events: `checkout.session.completed`, `customer.subscription.deleted`
3. Copy the signing secret → paste as `STRIPE_WEBHOOK_SECRET` in Netlify env vars

---

Page Routes

Route	Access	Description
`/`	Public	Landing page
`/login`	Public	Sign in
`/signup`	Public	Create account
`/free`	Public	Free board submission
`/upgrade`	Public	Pricing + Stripe checkout
`/investors`	Public	Investor contact form
`/account`	Members only	Member hub + billing portal
`/workout`	Members only	Today's daily workout
`/settings`	Members only	Profile, training, SMS prefs, password
`/success`	Members only	Post-payment confirmation
`/admin`	Admin role only	Members, free board, waitlist dashboard

What Still Needs to Be Done

Item	Status	Notes
Supabase SQL schema	**Pending**	Run the SQL above in Supabase dashboard
Stripe account + keys	**Pending**	Create products/price in Stripe, paste keys in `.env`
Stripe webhook	**Pending**	Register after first deploy
Twilio SMS	Optional	Leave blank to skip — reminders function won't fire without a phone number

Admin account	**Pending**	After first signup, run `update profiles set role = 'admin' where email = 'your@email.com'` in Supabase SQL editor

---

Making a User an Admin

After creating an account through `/signup`, run this in the Supabase SQL editor:

```
update profiles set role = 'admin' where id = (
  select id from auth.users where email = 'your@email.com'
);
```

Admin users can access `/admin` and see all members, free board posts, and waitlist signups. They also get all Premium features unlocked automatically.